Coding weeks 2025

Section 2048: Dino Run

Groupe 33 (Expédition 33): **LENHARDT Arthur** (arthur.lenhardt@student-cs.fr), **FOURNIER Hugo** (hugo.fournier@student-cs.fr), **NACIRI Deyaa-Eddine** (deyaa-eddine.naciri@student-cs.fr), **HAMDOUN Zakaria** (zakaria.hamdoun@student-cs.fr), **BENJELLOUN Ghali** (ghali.benjelloun@student-cs.fr), **HAKIMI Arthur** (arthur.hakimi@student-cs.fr)

Inspiration



Démonstration

MVP

1. Coder les entités de base (Dino, obstacles, fond)
2. Implémenter les interactions entre les objets et le joueur
3. Gestion de la difficulté du jeu
4. Système de scoring

Sprints

Sprint 1 — Base du jeu :

- **Création du personnage**
- **Saut et accroupissement**
- **Obstacles de base**

Sprint 2 — Entretien du MVP:

- **Tests et débogage**
- **Amélioration des collisions**

Sprints

Sprint 3 — Refactoring, thèmes & interface:

- **Refactoring complet de main.py**
- **Correction de bugs**
- **Ajout du sol**
- **Sons d’action (saut, course…)**
- **Amélioration du GUI**

Sprints

Sprint 4 — Pistolet, nouvel ennemi & menus:

- **Ajout du pistolet**
- **Mécanisme de tir**
- **Choix des thèmes depuis les menus**
- **Suite du refactoring: Ajout du module config, du gestionnaire des Assets et des sons**
- **Création de l’arrière-plan pour chaque thème**

Sprints

Sprint 5 — Menus avancés, difficulté & polish:

- **Menu des skins finalisé**
- **Sprites de skins créés**
- **Sprite du pistolet**
- **Logique des thèmes finalisée**
- **Corrections de bugs**

Sprints

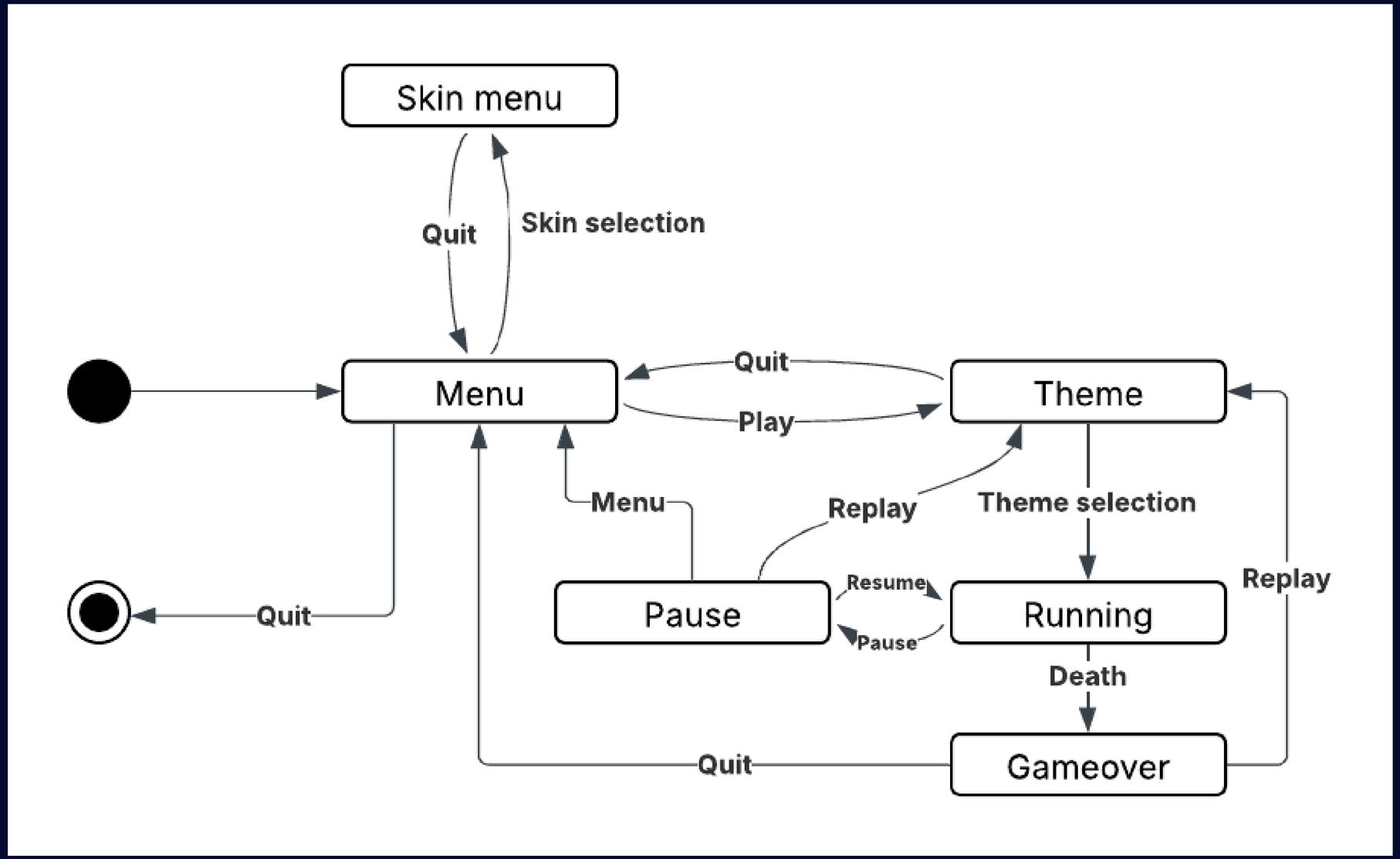
Sprint 6 — Tests, nouvel ennemi & finitions:

- **Testing global & tuning des paramètres**
- **Ajout d’un nouvel ennemi**
- **Sound Manager intégré partout**
- **Polishing & corrections finales**

Participation de chacun

- Arthur H. : Obstacles
- Arthur L. : Sonographie
- Ghali : Graphismes
- Deyaa : Menus / Background / Thèmes
- Hugo : Player
- Zakaria : Structure globale/ Modules Config et States / Debug

Logique des menus



```
.
├── assets/
│   ├── Backgrounds
│   ├── skins
│   ├── obstacles
│   └── ...
├── Musiques
├── Sons
├── config/
│   ├── assets.py
│   ├── settings.json
│   └── settings.py
├── scripts/
│   ├── states/
│   │   ├── play_state
│   │   └── ...
│   ├── player.py
│   ├── game.py
│   ├── bullet.py
│   ├── sound_management.py
│   └── Tests
├── main.py
├── README.md
└── requirements.txt
```

Refactoring

```python
bg = background(img = assets.backgrounds['background'])
player = Player(settings, assets)

menu_principal = MainMenu(WIDTH,HEIGHT,img = os.path.join("assets", "Backgrounds", "Menu", "menubg_test.jpg"))
menu_option = OptionMenu(WIDTH,HEIGHT,img = os.path.join("assets", "Backgrounds", "Menu", "menubg_test.jpg"))
menu_game_over = GameOverMenu(WIDTH,HEIGHT)
menu_pause = PauseMenu(WIDTH,HEIGHT)
jeu_actif = False
en_cours = True
etat_menu = "main"
boutons = {}

# Lancement initial de la musique du menu
play_music(music_menu_file)


while en_cours:


    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            en_cours = False
            settings.save_settings()

        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_ESCAPE:
                if jeu_actif :
                    jeu_actif = False
                    etat_menu = 'pause'
                    play_music(music_menu_file)
                elif etat_menu == 'option' :
                    etat_menu = 'main'
                elif etat_menu == "main" :
                    en_cours = False
                elif etat_menu == 'pause':
                    etat_menu = 'main'
                    jeu_actif = True
                    play_music(music_game_file)

        if event.type == pygame.KEYUP:
            if jeu_actif:
                if event.key == pygame.K_UP:
                    player.end_jump()
        if event.type == pygame.MOUSEBUTTONDOWN:
            if event.button == 1: #clic gauche
                if not jeu_actif :
                    if "start" in boutons and boutons["start"].collidepoint(event.pos):
                        jeu_actif = True
                        play_music(music_game_file)
                    if "quitter" in boutons and boutons["quitter"].collidepoint(event.pos):
                        en_cours = False
                        settings.save_settings()
                    if "options" in boutons and boutons["options"].collidepoint(event.pos):
                        etat_menu = "option"
                    if "quitter_over" in boutons and boutons["quitter_over"].collidepoint(event.pos):
                        etat_menu = "main"
                        jeu_actif = False
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

main.py

main.py > ...

```python
from scripts.game import Game

if __name__ == "__main__":
    game = Game()
    game.run()
```

Class Game

```python
class Game:
    """Classe pour gerer le jeu
    """
    def __init__(self):
        pygame.init()
        pygame.mixer.init()

        self.settings = Settings()
        self.width, self.height= self.settings.game_width, self.settings.game_height
        self.screen = pygame.display.set_mode((self.width, self.height))

        # self.screen = pygame.display.set_mode((self.settings.width, self.settings.height))
        pygame.display.set_caption(self.settings.game_title)
        self.clock = pygame.time.Clock()

        # Chargement des assets
        self.assets = AssetManager()

        # Bool pour indiquer si le jeu est en cours
        self.running = True

        self.player = None
        # Menu
        self.current_state = MenuState(self)
```

Class Game

```
32      def run(self):
33          while self.running:
34              # Collecte des evenements
35              events = pygame.event.get()
36              for event in events:
37                  if event.type == pygame.QUIT:
38                      self.running = False
39                      self.settings.save_settings()
40
41              # Logique de l'etat actuel
42              self.current_state.handle_input(events)
43              self.current_state.update()
44
45              # Dessiner
46              self.current_state.draw(self.screen)
47
48              # Actualiser le jeu
49              pygame.display.update()
50              self.clock.tick(self.settings.game_fps)
51
52          # Sauvegarde des parametres et high_score
53          self.settings.save_settings()
54          pygame.quit()
```

Class State

scripts > states > state.py > State

C:\Users\dimcp\Desktop\Coding\coding_week\project-expedition-33\scripts • Contains emphasized items

```python
class State:
    """Class abstraite modelisant un etat du jeu (menu, jeu, ...)
    """

    def __init__(self, game):
        self.game = game
        # Etat precedent, ca nous sera utile pour faire une pause scre
        self.prev_state = None

    def update(self):
        """Fonction principale gerant l'etat
        """
        pass

    def draw(self, surface):
        """S'occupe de la partie graphique de l'etat
        """
        pass

    def handle_input(self, events):
        """Gestion des evenements pygame
        """
        pass

    def enter_state(self):
        """Applee quand on entre dans cet etat"""
        pass

    def exit_state(self):
        """Applee quand on quitte cet etat"""
        pass

    def desc(self):
        pass
```

Settings

Mémoire permanente

settings.json M

config > {} settings.json > { } player > abc skin

```json
{
    "game": {
        "fps": 60,
        "title": "Expedition 33",
        "high_score": 183,
        "ground_y": 650,
        "game_speed": 14.90567,
        "width": 1280,
        "height": 720,
        "mode": "Facile",
        "acceleration": 0.5,
        "vmax": 40
    },
    "constants": {
        "gravity": 0.8
    },
    "player": {
        "jump_power": -20,
        "vitesse_limite": 15,
        "pos_initiale": [
            200,
            1000
        ],
        "skin": "snow"
    }
}
```

Mémoire centrale

```python
class Settings:
    """
    Classe permettant de gerer les parametres efficacement
    """
    def __init__(self):
        # Parametres par defaut

        # ------ Game ----------------
        self.debug_mode = False
        self.game_fps = 60
        self.game_title = "Expedition 33"
        self.game_high_score = 000
        self.game_ground_y = 600
        self.game_game_speed = 10
        self.game_mode = 'Facile'
        self.theme = "foret"
        self.game_acceleration = 0.5
        self.game_vmax = 40
        self.game_width = 1280
        self.game_height = 720

        # ------ Player ----------
        self.player_jump_power = -16
        self.player_vitesse_limite = 15
        self.player_pos_initiale = (200, 500)
        self.player_skin = 'forest'

        # ------ Constants -------------
        self.constants_gravity = 0.8

        # Chargement des parametres
        self.load()
```

```python
def load(self):
    """Extrait les parametres a partir du fichier settings.json
    """
    try:
        with open('config/settings.json', 'r') as f:
            data = json.load(f)

            # Game settings
            data_game = data.get('game', {})
            self.game_fps = data_game.get('fps', self.game_fps)
            self.game_title = data_game.get('title', self.game_title)
            self.game_high_score = data_game.get('high_score', self.game
            self.game_ground_y = data_game.get('ground_y', self.game_gro
            self.game_game_speed = data_game.get('game_speed', self.game
            self.game_mode = data_game.get('mode', self.game_mode)
            self.game_acceleration = data_game.get("acceleration", self.
            self.game_vmax = data_game.get("vmax", self.game_vmax)
            self.theme = data_game.get("theme",self.theme)
            self.game_width = data_game.get("theme",self.game_width)
            self.game_height = data_game.get("theme",self.game_height)

            # Player settings
            data_player = data.get('player', {})
            self.player_jump_power = data_player.get('jump_power', self.
            self.player_vitesse_limite = data_player.get('vitesse_limite
            self.player_pos_initiale = data_player.get('pos_initiale', s
            self.player_skin = data_player.get('skin', self.player_skin)
```

Obstacles et power-up

- Ajout des graphismes :





- Règle pour l’apparition:

```
if len(obstacles) == 0:
    nb = randint(0, 2)
    if nb == 0:
        obstacles.append(Oiseau())
    if nb == 1:
        obstacles.append(SmallCactus())
    if nb == 2:
        obstacles.append(LargeCactus())
```

```
if self.spawn_timer <= 0:
    self.spawn_obstacle()
    # Reinitialiser l'horloge aleatoirement
    self.spawn_timer = random.randint(self.min_spawn_time, self.max_spawn_time)
```

```
patterns.append([('oiseau', 400), ('oiseau', 0)])
```

- Condition de défaite:

```
collisions = pygame.sprite.spritecollide(self.player, self.obstacle_manager.obstacles, False)
powerups = pygame.sprite.spritecollide(self.player, self.obstacle_manager.powerups, True) # True pour supprimer le pistolet
if collisions:
    if not self.game.settings.debug_mode:
        # Entrer dans etat game_over
        self.game.current_state = GameOverState(self.game, final_score=self.score, prev_state=self)
```

```python
    patterns.append([('cactus', 0)])

    # --- NIVEAU 1: RHYTHME (Speed 14 - 17) ---
    # Motifs basiques
    if 14<= self.settings.game_game_speed <= 17:
        # double cactus
        patterns.append([('cactus', 380), ('cactus', 0)])

        # Oiseau + cactus
        patterns.append([('oiseau', 400), ('cactus', 0)])

    # --- NIVEAU 2: PRECISION (Speed 17 - 21) ---
    # Plus de rapidite
    if self.settings.game_game_speed >= 17:
        # Cactus puis frog
        patterns.append([('cactus', 520), ('jumping_enemy', 0)])

        # "Duck and Cover": Deux oiseaux
        # Duck -> Jump
        patterns.append([('oiseau', 400), ('oiseau', 0)])

    # --- NIVEAU 3: Pieges (Speed 21 - 24) ---
    # Mental load increases. You have to identify threats quickly.
    if self.settings.game_game_speed >= 21:
        # "Le MUR" 3 cactus
        patterns.append([('cactus', 500), ('cactus', 70), ('cactus', 0)])

        # Frog + bird
        patterns.append([('jumping_enemy', 450), ('oiseau', 0)])
```

Player

Ajout de la logique:

-saut

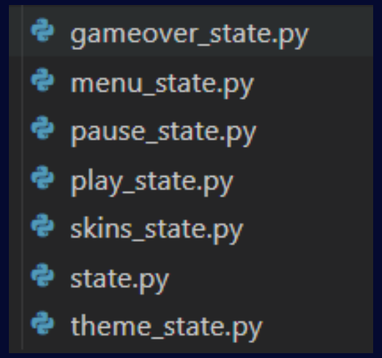
-s’accroupir

-mécanisme de tir

-gestion des entrées

Menu

- Organisés en états




```python
class MenuState(State):
    def __init__(self, game):
        super().__init__(game)
        # Initialisation de l'arrière plan
        self.game = game
        self.bg_img = game.assets.backgrounds['menu']['main']
        if self.bg_img:
            self.bg_img = pygame.transform.scale(self.bg_img, (game.width, game.height))
        # Initialisation du son
        self.sound = SoundManager()
        # Définition de la police
        self.font = pygame.font.Font(None, 60)
        self.font_text = pygame.font.Font(None, 45)
        # Creation des boutons
        cw, ch = game.width // 2, game.height // 2
        self.btn_play = Button(cw//2,7*ch//4 , "COMMENCER", self.font,(240,230,140), (255,215,0))
        self.btn_skins = Button(cw,7*ch//4 , "SKINS", self.font,(240,230,140), (255,215,0))
        self.btn_quit = Button(3*cw//2,7*ch//4, "QUITTER", self.font,(240,230,140), (255,215,0))


        #Musique
        self.sound.play_music("accueil")
```

```python
def text(self,texte,font,couleur,x,y,fenetre):
    txt_to_img = font.render(texte,True,couleur)
    rect = txt_to_img.get_rect()
    rect.center  = (x,y)
    fenetre.blit(txt_to_img,rect)


def update(self):
    # Vérifie si un des boutons a été survolé
    mouse_pos = pygame.mouse.get_pos()
    self.btn_play.update(mouse_pos)
    self.btn_skins.update(mouse_pos)
    self.btn_quit.update(mouse_pos)


def draw(self, surface):
    # Dessine l'arrière plan
    if self.bg_img:
        surface.blit(self.bg_img, (0,0))
    else:
        surface.fill("gray")
    # Dessine les boutons et meilleur score
    self.btn_play.draw(surface)
    self.btn_skins.draw(surface)
    self.btn_quit.draw(surface)
    self.text("High Score : " + str(self.game.settings.ga
```

```python
class Button():
    def __init__(self, x, y, text, font, color_base, color_hover): ···

    def update(self, mouse_pos): ···


    def draw(self, screen): ···

    def is_clicked(self, event): ···

class ImgButton():
    def __init__(self, x, y, color_base, color_hover, image_surface,is_selected = False): ···

    def update(self, mouse_pos): ···

    def outline(self,screen,is_selected): ···


    def draw(self, screen): ···

    def is_clicked(self, event): ···
```

```python
def handle_input(self, events):
    # Collecte des évènements
    for event in events:
        # Logique du bouton play
        if self.btn_play.is_clicked(event):
            new_state = ThemeState(self.game)
            self.game.current_state = new_state
            self.sound.play_sound('bouton_confirm')

        # Logique du boutton quitter
        if self.btn_quit.is_clicked(event):
            self.game.running = False

        # Logique du bouton skins
        if self.btn_skins.is_clicked(event):
            new_state = SkinState(self.game)
            self.game.current_state = new_state
            self.sound.play_sound('bouton_confirm')
```

Arrière plan défilable

```python
import pygame as pygame
from pygame.locals import *
import math

pygame.init()

class background():
    def __init__(self,settings,img = None):
        self.img = img
        self.settings = settings
        self.img = pygame.transform.scale(self.img, (self.settings.game_width, self.settings.game_height)).convert()
        self.width = self.img.get_width()
        self.defilement_horizontal = 0
        self.pavage = math.ceil(self.settings.game_width / self.width) + 1

    def maj(self):
        self.defilement_horizontal -= self.settings.game_game_speed
        if self.defilement_horizontal <= -self.width:
            self.defilement_horizontal += self.width

    def affiche(self,fenetre):
        for i in range(self.pavage):
            x_position = (self.width * i) + self.defilement_horizontal
            fenetre.blit(self.img, (int(x_position), 0))
```

- Initialisation
- Mise à jour
- Affichage

Thèmes

création des boutons de choix

```
themes = ["forest","desert","snow"]
for theme in themes:
    image = self.game.assets.backgrounds['game'][theme]
    cadre = pygame.transform.scale(image, (self.choix_l, self.choix_h))
    self.choix[theme] = cadre


# Bouttons images
self.theme_1 = ImgButton(image_surface=self.choix["snow"], x = self.largeur//4,y = 5*self.hauteur//8,color_base=(255, 215, 0), color_hover=(255, 255, 255))
self.theme_2 = ImgButton(image_surface=self.choix["forest"], x = self.largeur//2,y = 5*self.hauteur//8,color_base=(255, 215, 0), color_hover=(255, 255, 255))
self.theme_3 = ImgButton(image_surface=self.choix["desert"], x = 3*self.largeur//4,y = 5*self.hauteur//8,color_base=(255, 215, 0), color_hover=(255, 255, 255))
```

- **gestion du choix**

```
def handle_input(self, events):
    for event in events:
        if self.theme_1.is_clicked(event):
            theme = "snow"
            self.game.settings.game_ground_y = 620
            new_state = PlayState(self.game, theme)
            self.game.current_state = new_state
```

Manager

```
def load_all(self):
    print("[AssetManager][Info] Chargement des assets ...")

    # --- Player ---
    skins = ['forest', 'snow', 'desert']
    for skin in skins:···

    # --- Backgrounds ---

    # Game backgrounds
    self.backgrounds['game']['snow'] = self.load_image(['backgrounds', 'game'], 'snow.png', transparent=False)
    self.backgrounds['game']['forest'] = self.load_image(['backgrounds', 'game'], 'forest.png', transparent=False)
    self.backgrounds['game']['desert'] = self.load_image(['backgrounds', 'game'], 'desert.png', transparent=False)

    # Menu backgrounds
    self.backgrounds['menu']['main'] = self.load_image(['backgrounds', 'menu'], 'main_menu_background.png', transparent=False)
    self.backgrounds['menu']['theme'] = self.load_image(['backgrounds', 'menu'], 'menus_background.png', transparent=False)
    self.backgrounds['menu']['skins'] = self.load_image(['backgrounds', 'menu'], 'menus_background.png', transparent=False)

    # --- Obstacles ----
    # Statique
    # Cactus
    self.obstacles['statique']['cactus'] = []
    for i in range(6):···

    # A completer


    # Dynamique
    self.obstacles['dynamique']['oiseau'] = [self.load_image(['obstacles', 'dynamique', 'oiseau'], f'1.png'),···

    # --- Other ----
    self.other['bullet'] = self.load_image(['other'], 'bullet.png')
    self.other['gun'] = self.load_image(['other'], 'gun.png')
```

Graphisme

- Utilisation du module AssetManager pour charger les sprites
- Animations : player, oiseaux, pistolet, ennemi
- Création de skins : player, pistolet, ennemi. Consistance sur la taille en pixel notamment pour la hitbox, méthode établie et utilisée.
- Création backgrounds (défilable)

Sonographie

- Enregistrement de sons pour le bruitage du jeu : Jump, Mort Dinosaure, Tir, Récupération gun
- Choix de musiques : une pour l’écran d’accueil, une pour le jeu et une pour l’écran game over
- Implémentation des sons et de la musique dans le code
- Création d’une classe SoundManager pour faciliter l’implémentation des sons dans le jeu et alléger le script main
- Ajout de sons supplémentaires

Sonographie

```python
class SoundManager:
    """
    Gère le chargement et la lecture de tous les sons et musiques du jeu.
    """
    def __init__(self):
        # S'assurer que pygame.mixer est initialisé
        if not pygame.mixer.get_init():
            pygame.mixer.init()
        self.sounds = {}
        self.musics = {
            "troll" : os.path.join("Musiques","Musique_thème_troll__Badali-Badala_.mp3"),
            "classique" : os.path.join("Musiques", "Musique_thème_classique_Night_Drifting.
            "gameover" : os.path.join("Musiques", "Musique_game_over.mp3"),
            "accueil" : os.path.join("Musiques", "Musique_écran_acceuil.mp3")}
        self.current_music = ""
        self.sound_paths = {
            "jump": os.path.join("sons", "jump.wav"),
            "dino_death": os.path.join("sons", "mort.wav"),,
            "gun": os.path.join("sons","tir.wav"),
            "recup" :  os.path.join("sons","recuperer.wav"),
            "bouton_confirm": os.path.join("sons","bouton_confirmation.ogg"),
            "drop": os.path.join("sons","drop.ogg"),
            "drop1": os.path.join("sons","drop1.ogg"),
            "confirm": os.path.join("sons","confirm.ogg"),
            "hit_cactus": os.path.join("sons","hit_cactus.ogg"),
            "hit": os.path.join("sons","hit.ogg"),
            "pause": os.path.join("sons","pause.ogg")
        }

        self.load_sounds()
```

```python
def load_sounds(self):
    """Charge tous les effets sonores (utilisent des Channel)."""
    print("Chargement des effets sonores (SFX)...")
    for name, path in self.sound_paths.items():
        try:
            # Charger le son
            self.sounds[name] = pygame.mixer.Sound(path)
            print(f"- Son '{name}' chargé : {path}")
        except pygame.error as e:
            print(f"Erreur de chargement du son '{name}' à '{path}': {e}")
```

```python
def play_sound(self, name):
    """Joue un effet sonore une seule fois."""
    if name in self.sounds:
        self.sounds[name].play()
    else:
        print(f"Erreur : Son SFX '{name}' non trouvé.")
```

```python
def play_music(self, name, loops=-1):
    """
    Charge et joue une musique en boucle (loops=-1 pour une boucle infinie).
    Évite de recharger la même musique.
    """
    if name in self.musics:
        file_path = self.musics[name]

        # Si c'est déjà la musique jouée, ne fait rien
        if self.current_music == file_path:
            return

        # Sinon, arrête l'ancienne, charge et joue la nouvelle
        self.stop_music()
        try:
            pygame.mixer.music.load(file_path)
            pygame.mixer.music.set_volume(0.3)
            pygame.mixer.music.play(loops=loops)
            self.current_music = file_path
            print(f"Musique '{name}' lancée : {file_path}")
        except pygame.error as e:
            print(f"ÉCHEC DE CHARGEMENT OU LECTURE DE LA MUSIQUE {file_path}: {e}")
            self.current_music = ""
    else:
        print(f"Erreur : Nom de musique '{name}' non enregistré.")

def stop_music(self):
    """Arrête la musique en cours."""
    pygame.mixer.music.stop()
    self.current_music = ""
```

Utilisation de l’IA

- Documentation bibliothèque pygame : implémentation de sons ou de musiques, création d’une interface graphique, gestion de la vitesse et du temps
- Recherche d’idées pour les graphismes et pour les thèmes

Utilisation du Git

- Travail à plusieurs sans conflit grâce au système de branches : chacun travaillait sur une fonctionnalité ou une partie du code avant de merge
- Documentation de l’évolution du projet grâce au message de commit
- Utilisation de protection sur la branche main pour s’assurer qu’elle ne soit pas sujette au bug

feature-jumping-enemy

main **features**

feature-pistolet

Documentation

Code clair, bien documenté et structuré: Commentaires, docstrings...

3 fichier README.md

- *Readme.md global du projet*
- *Un autre pour le package states*
- *Un dernier pour le package config*

Perspectives d’amélioration

- Ajout de nouveaux ennemis, nouvelles armes
- Amélioration de la cohérence hitbox graphismes
- Avoir d’autres genres de power-ups
- Ajout d’un multijoueur